

Week 3 Progress Report: Closing the Gap on DET-Phase Calibration

Prepared by: Shalin Bhavsar

Date: June 27 - July 03, 2026

Supervisor: Dr. Yi Zheng, Arizona State University

Summary

Following up on the last report, I focused on closing the gap between my DET-phase offline calibration results and the paper’s Table 1/Table 2 numbers, per your note to improve performance before moving to the next phase. Along the way I found and fixed a real bug in my jump-start splitting logic, and ran an investigation into an unexpected result that turned out to be a measurement artifact rather than a real problem. Details below.

1 Bug found and fixed: Jump 40 and Jump 80 were not actually different conditions

While reviewing results, I noticed Jump 40 and Jump 80 were returning byte-identical metrics for both item types. Root cause: at my previous session count (1,500), the average number of post-split responses available per pilot item worked out to roughly 9.6 — far below even $R = 20$, let alone $R = 40$ or $R = 80$. So “first R responses per pilot item” was silently capped by how much data existed, not by R itself. The three jump-start conditions weren’t actually testing different amounts of pilot data.

Fix: increased simulated sessions to 25,000, which gives enough post-split volume per pilot item to reach $R = 80$ with margin for the item bank size that was current at the time. I added a diagnostic check to the split logic that flags this automatically in any future run if a similar data-starvation issue occurs – see the caveat in Section 2, because that check fired on this very run.

2 DET-phase results: substantial improvement, gap not fully closed

Three things changed between this run and the previous measurement, not just the session count: the session count (1,500 $\rightarrow$ 25,000, the fix above), the item bank size (up to the full real-DET sizes, 3,290 Y/N Vocab / 585 ViC, not 800/400), and the backend actually used for this run was the hand-built ensemble, not AutoGluon – `config.py` sets `DET_BACKEND_OVERRIDE = "ensemble"` for faster iteration, which overrides the project default of `BACKEND = "autogluon"`. So this isn’t a clean before/after on the jump-start fix alone, and the backend difference in particular is worth keeping in mind when comparing against the paper’s numbers, since their results use AutoGluon throughout. Here’s the before/after on cold-start (the paper’s hardest use case):

Metric	Previous report	Current	Paper
Y/N Vocab Cold Pearson	0.441	0.745	0.785
ViC Cold Pearson	0.577	0.656	0.742

Full current results across all six DET conditions:

Item Type	Split	Pearson	Paper	Gap
Y/N Vocab	Cold	0.745	0.785	0.04
Y/N Vocab	Jump 20	0.801	0.936	0.14
Y/N Vocab	Jump 40	0.726	0.955	0.23
Y/N Vocab	Jump 80	0.834	0.971	0.14
Y/N Vocab	Warm 05-08	0.841	0.987	0.15
Y/N Vocab	Warm 05-15	0.830	0.987	0.16
ViC	Cold	0.656	0.742	0.09
ViC	Jump 20	0.803	0.954	0.15
ViC	Jump 40	0.855	0.978	0.12
ViC	Jump 80	0.874	0.989	0.12
ViC	Warm 05-08	0.929	0.998	0.07
ViC	Warm 05-15	0.932	0.998	0.07

Cold-start improved a lot: the Y/N Vocab gap closed from 0.344 ($0.785 - 0.441$) to 0.040 ($0.785 - 0.745$), and ViC’s closed from 0.165 ($0.742 - 0.577$) to 0.086 ($0.742 - 0.656$) – both now the *smallest* gap of the six conditions for their item type, not the largest. That’s the opposite of what I expected going in, since the paper frames cold-start as the hardest case, and it means I had the direction backwards in my head while this run was going. Jump-start and warm-start now show the larger, less consistent gaps (0.07–0.23), with Y/N Vocab Jump 40 the single worst point (0.23).

I don’t yet have a clean explanation for why jump/warm-start are proportionally further from the paper than cold-start is here, and I don’t want to guess at one without more evidence. Two candidates I’d want to rule in or out before the next report: the ensemble-vs-AutoGluon backend difference noted above (the backend-comparison run showed ensemble converging cleanly on warm-start while AutoGluon didn’t, in an unrelated small-scale test – not obviously predictive of this direction, but it’s the most concrete backend-related lead I have), and the much larger item bank size interacting differently with jump/warm-start’s larger training sets than with cold-start’s feature-only case. I’d rather flag this honestly as unresolved than pick one and present it as settled.

One caveat on Y/N Vocab Jump 80 specifically: the diagnostic check added in Section 1 fired on this run – 0 of 1,645 pilot items actually reached 80 post-split responses (0% coverage), meaning this condition is not actually testing $R = 80$ at the current item bank size and session count; it’s silently falling back to whatever pilot data exists, the same failure mode as the original bug, just not fully closed yet at this larger item bank size. I’d treat the Y/N Vocab Jump 80 row above as unreliable until I either raise `N_DET_SESSIONS` further or interpret it with that caveat attached.

3 Item bank size investigation: resolved, and the mechanism does hold

I wanted to understand what’s still limiting cold-start performance, so I ran the item-bank-size sweep (400 and 1,600 items — I dropped the 100-item point to save runtime at this larger scale) at the paper’s own comparison point of 10,000 sessions. The first result was concerning: 1,600 items scored *worse* than 400 (Pearson 0.653 vs. 0.696), the opposite of the paper’s reported $0.58 \rightarrow 0.83$ pattern.

I tested two hypotheses in sequence:

- **AutoGluon fit-time budget too short at larger item counts?** Reran the 1,600-item case with the time budget raised from 60s to 150s per fit. Result was essentially unchanged ($0.653 \rightarrow 0.653$). Ruled out.
- **Measurement noise from a too-small evaluation set?** The sweep had been using only 50 held-out items and a single random seed (the paper uses 1,000). Reran with 200 held-out items and 3 seeds per item-bank size: 400 items gave 0.384 ± 0.055 , 1,600 items gave **0.680 ± 0.094** . The direction flipped

back to match the paper once measured properly — the original “inversion” was sampling noise from an undersized, single-run evaluation, not a real problem with the calibration approach.

Net takeaway: the item-bank-size mechanism the paper describes does hold in this implementation. The DET run’s cold-start results are evaluated against the full post-split item bank (up to 3,290 items for Y/N Vocab, 585 for ViC — pilot and operational combined), not the 50-item, single-seed sample that produced the misleading sweep result, so I trust those DET numbers more than that early sweep result.

4 Note on a correction I made to my own code comments

While reviewing my backend-comparison script, I found a comment I’d written claiming AutoGluon gave a cold-start Pearson of 0.510 at a 60-second time budget. That number doesn’t appear in any actual run output I have — the real comparison run shows 0.268 at that setting, a modest improvement over the ensemble backend’s 0.228, not the “roughly doubling” I’d written. I corrected the comment and the mirrored claim in two other files before it went anywhere. Flagging this here for transparency since it shaped an earlier backend decision, even though the actual DET numbers above were generated after the correction and aren’t affected by it.

Next steps

- Would like your read on the gap pattern above: it’s now jump-start and warm-start that are furthest from the paper (0.07–0.23), not cold-start (0.04–0.09), which is the reverse of what I expected. Open to your take on whether that’s worth chasing now or is more likely a byproduct of running this on the ensemble backend instead of AutoGluon.
- Will re-run Y/N Vocab Jump 80 with a higher session count to fix the data-starvation issue flagged above before trusting that row.
- Will look into whether the backend swap (ensemble vs. AutoGluon) explains the jump/warm-start gap pattern, using `run_backend_comparision.py` at the current item bank scale.
- Ready to move to the next phase whenever you’d like, pending your view on the above.